Node.js Developer Roadmap

🟢 Phase 1 — Express Fundamentals

📚 Required Knowledge

A junior must understand:

- What is Express?
- Creating a server
- Routing (GET, POST, PUT, DELETE)
- Request & Response objects
- Middleware
- `next()` function
- Handling JSON
- Status codes
- realTime (Socket IO)
- Basic error handling

🛠 Task Requirements

✅ Task 1 — Build a Basic API Server

Create an Express server that:

- Runs on port 3000
- Has the following routes:
  - `GET /` → returns "API is running"
  - `GET /users`
  - `POST /users`

    - `PUT /users/:id`
    - `DELETE /users/:id`
- Uses `express.json()` middleware
- Returns proper status codes
- Handles invalid routes (404 middleware)
- make realtime logic for anyting in your project

✅ Task 2 — Middleware Practice

Create:

- A custom logging middleware
- A middleware that blocks requests without a specific header
- A global error handling middleware

🟢 Phase 2 — REST API & Project Structure

📚 Required Knowledge

- REST principles
- Separation of concerns
- Controllers
- Routes folder
- Modular structure
- Environment variables (.env)
- Using nodemon

✅ Task 3 — Refactor Project Structure

Restructure the project into:

```
src/
  controllers/
  routes/
  middlewares/
  services/
  app.js
server.js
```

Requirements:

- Routes separated from logic
- Controllers handle business logic
- Middleware separated
- Use `.env` for PORT

🟢 Phase 3 — Database (MongoDB + Mongoose)

📚 Required Knowledge

- Connecting to MongoDB
- Mongoose schemas
- CRUD operations
- Validation
- Async/Await
- try/catch
- Relationships in MongoDB

- populate

🛠️ Task Requirements

✅ **Task 4 — Connect to Database**

- Connect to MongoDB
- Create model:
- Implement full CRUD
- Validate required fields
- Handle duplicate email error (exmple)

🟢 **Phase 4 — Authentication & Security**

📚 Required Knowledge

- Hashing passwords (bcrypt)
- JWT authentication
- Login / Register flow
- Protected routes
- Auth middleware
- CORS
- Helmet
- Environment variables

🛠️ Task Requirements

✅ Task 5 — Authentication System

Build:

- POST /register
- POST /login
- Hash password before saving
- Generate JWT on login
- Protect route `/profile`
- Middleware to verify token
- Handle invalid/expired token

🟢 Phase 5 — Error Handling & Best Practices

📚 Required Knowledge

- Centralized error handler
- Custom error class
- Proper HTTP status codes
- Avoid blocking code
- Async error wrapper

🛠 Task Requirements

✅ Task 6 — Advanced Error Handling

- Create custom `AppError` class
- Global error middleware
- Handle:

    - Validation errors
    - Cast errors
    - JWT errors
- Return consistent error response format

🟢 **Phase 6 — Clean Architecture Basics**

📚 Required Knowledge

- MVC pattern
- Service layer
- Clean code principles
- Reusable logic
- Avoid fat controllers

🛠 Task Requirements

✅ **Task 7 — Refactor Using Service Layer**

- Move business logic to services
- Controllers should:
    - Receive request
    - Call service
    - Return response